

FECAP — Fundação Escola de Comércio Álvares Penteado

Projeto Interdisciplinar: Inteligência Artificial

ASA · Agente para o Estudante

Entrega 1 — Concepção da solução e prova de viabilidade

Grupo: EriTech +

Integrantes: Bruno Eduardo da Silva Pinho · Eric Victor Bittu Silva · Vivian Kaori Umaki · Yanko Lee

Curso: Ciência da Computação — 5º Semestre 2026

Professor orientador: Rafael Diogo Rossetti

Orientadores complementares: Marcelo de Moura Amorim · Marcos Minoru Nakatsugawa · Rodnil da Silva Moreira Lisboa · Rodrigo da Rosa

Repositório: github.com/2026-2-NCC5/Projeto3

Data de entrega: 25 de setembro de 2026, até 23:59:56

1. Problema, usuários, jornada e agente selecionado

1.1 Contexto

A Área do Sucesso Alvarista (ASA) acolhe e acompanha os estudantes da FECAP ao longo de sua jornada acadêmica, reunindo uma equipe multidisciplinar e serviços que vão de biblioteca e financiamento a empregabilidade e carreira. O ASA não é uma área de atendimento formal, mas na prática acaba respondendo dúvidas e acompanhando a trajetória dos estudantes.

1.2 Problema

O levantamento de descoberta feito com a equipe do ASA identificou que a informação institucional existe, mas não resolve o problema do estudante no momento em que ele precisa dela. Os principais sintomas encontrados foram:

- **Canais fragmentados:** presencial, e-mail, WhatsApp e portal do aluno não se comunicam entre si.
- **Filas e espera:** sem resposta imediata, o estudante troca de canal e reinicia a espera em outro.
- **Informação espalhada:** regulamentos, manual do aluno, Moodle e documentos de várias áreas ficam salvos separadamente na rede.
- **Documentos longos demais:** o estudante quer a resposta em poucas linhas, não um PDF de dez páginas.
- **Inconsistência de informação:** dois atendentes podem responder a mesma pergunta de formas diferentes, o que corrói a confiança do estudante na informação oficial.
- **Resposta genérica não resolve:** sem considerar a situação real do estudante, a orientação dada não chega ao caso concreto dele.

1.3 Usuários

Perfil	Necessidade principal	Como usa o agente
Estudante	Obter respostas rápidas e confiáveis sobre procedimentos, prazos e serviços do ASA/FECAP.	Faz perguntas em linguagem natural via aplicação web e recebe resposta curta, com fonte e próximo passo.
Atendente do ASA	Reduzir volume repetitivo de perguntas simples e ter respostas padronizadas para apoiar seu próprio atendimento.	Consulta o mesmo agente ao atender um caso, garantindo resposta consistente com a que o estudante recebeu.
Administrador/equipe técnica	Manter a base de conhecimento atualizada, acompanhar qualidade das respostas e auditar o uso.	Cadastra, versiona e desativa documentos oficiais; acompanha dashboard e casos de abstenção.

1.4 Jornada do estudante (cenário principal)

1. O estudante acessa a aplicação web do ASA e faz login com seu perfil.
2. Ele digita uma dúvida em linguagem natural (ex.: “como eu tranco minha matrícula?”).
3. O agente busca nas fontes oficiais autorizadas (RAG) e, quando aplicável, cruza com a situação acadêmica do estudante (via integração com o TOTVS).
4. O agente devolve uma resposta curta, com passo a passo, fonte e data de atualização.
5. Se a confiança da busca for baixa, o agente se abstém de responder e encaminha para atendimento humano do ASA.
6. A interação é registrada no histórico, respeitando as permissões de acesso, para fins de auditoria e melhoria contínua.

1.5 Agente selecionado

Dentre os três agentes propostos no enunciado (Monitoramento de Estudantes, Pendências e Agente para o Estudante), o grupo EriTech + selecionou o Agente para o Estudante como escopo principal: um agente conversacional baseado em RAG (Retrieval-Augmented Generation) que orienta o estudante sobre serviços, procedimentos, documentos e canais oficiais da FECAP/ASA, respondendo sempre com base em conteúdo institucional autorizado e citando fonte e data de atualização.

1.6 MVP e valor esperado para o ASA

O MVP definido para a Entrega 1/2 cobre: recebimento de pergunta do estudante, busca semântica na base de conhecimento oficial, resposta com evidências (fonte, trecho, data), abstenção quando a confiança for baixa, e integração simulada (mock) com o TOTVS para personalizar a orientação à situação real do estudante.

O valor esperado para o ASA é reduzir o volume de perguntas repetitivas que hoje sobrecarregam os canais humanos, padronizar a resposta institucional (acabando com a inconsistência entre atendentes) e dar ao estudante uma resposta em segundos, com rastreabilidade de onde veio a informação.

1.7 Decisão de escopo: base de conhecimento desta entrega

O ASA disponibilizou ao grupo uma base de documentos institucionais para uso no projeto interdisciplinar. Após análise, o grupo identificou que essa base atende principalmente aos agentes de Monitoramento de Estudantes e de Pendências, com pouca aderência direta ao escopo do Agente para o Estudante.

Por isso, esta Entrega 1 usa como base de conhecimento os 20 documentos fictícios já modelados no repositório (seed_v2.sql), cobrindo os principais temas de atendimento do ASA (rematrícula, financeiro, trancamento, TCC, biblioteca, entre outros) com conteúdo plausível e estrutura idêntica à que será usada com documentos reais. A coleta e ingestão dos documentos oficiais reais do ASA, mais aderentes ao Agente para o Estudante, fica planejada para a Entrega 2, sem impacto na arquitetura já construída: o pipeline de ingestão (seção 3.3) e o endpoint de busca semântica já funcionam com qualquer conteúdo cadastrado em documentos_oficiais/documentos_chunks.

2. Requisitos, regras, critérios de aceitação, riscos éticos e proteção de dados

2.1 Requisitos funcionais (escopo do agente selecionado)

ID	Funcionalidade	Descrição
RF01	Autenticação e perfis	Acesso seguro conforme perfil (estudante, atendente ASA, administrador), com princípio do menor privilégio.
RF02	Perfil do estudante	Consolida apenas os dados autorizados necessários para responder à pergunta.
RF03	Importação e integração	Recebe documentos oficiais e dados acadêmicos autorizados (via mock do TOTVS), validando formato e origem.
RF06	Explicação e evidências	Toda resposta apresenta trecho recuperado, fonte, similaridade e data de atualização.
RF07	Recomendação de ação	Indica próximo passo e canal correto, sem executar decisões restritivas automaticamente.
RF08	Validação humana	Permite que o atendente confirme, ajuste ou complemente a orientação dada.

ID	Funcionalidade	Descrição
RF09	Alertas e fila de trabalho	Organiza os casos encaminhados ao ASA por prioridade e prazo, evitando notificações duplicadas ou excessivas.
RF10	Histórico e auditoria	Registra pergunta, resposta, confiança e encaminhamento, respeitando permissões de acesso.
RF15	Orientação ao estudante	Responde com base em conteúdo oficial, fornece passo a passo e encaminha para atendimento humano quando necessário.
RF16	Base de conhecimento	Permite cadastrar, versionar e desativar documentos oficiais usados pelo agente.
RF17	Feedback e melhoria	Coleta avaliação do estudante sobre a utilidade da resposta, para análise e melhoria contínua do agente.
RF18	API e publicação	Disponibiliza endpoint documentado (POST /api/perguntar) para inferência.

Os IDs seguem a numeração completa do backlog de requisitos do edital, compartilhado entre os três agentes. Os RFs não listados acima pertencem aos agentes de Monitoramento de Estudantes e Pendências, ou são extensões opcionais fora do escopo desta entrega.

2.2 Requisitos não funcionais

- **Segurança:** autenticação, chaves de serviço do Supabase mantidas apenas no backend, nunca expostas no frontend.
- **Privacidade e LGPD:** dados de estudantes usados no protótipo são fictícios; finalidade explícita e minimização de dados em qualquer consulta futura a dados reais.
- **Ética e não discriminação:** o agente nunca executa decisões punitivas, financeiras ou disciplinares; toda ação de impacto exige validação humana.
- **Explicabilidade:** toda resposta traz fonte, trecho e grau de confiança (similaridade), tornando o resultado auditável.
- **Confiabilidade:** mecanismo de abstenção quando a similaridade fica abaixo do limiar definido, evitando resposta sem evidência suficiente.
- **Desempenho:** tempo de resposta compatível com uso conversacional (poucos segundos), com fallback automático caso a chamada ao serviço de embeddings falhe.
- **Manutenibilidade:** código modular (API separada da base de dados e da lógica de vetorização), versionado no GitHub.
- **Portabilidade:** backend em Python, executável localmente e implantado na nuvem (Vercel + Supabase).

2.3 Regras de negócio

- O agente responde apenas com base em documentos oficiais cadastrados na base de conhecimento; não inventa procedimentos, prazos ou regras.
- Quando a similaridade da melhor evidência encontrada fica abaixo do limiar de confiança definido, o agente se abstém e encaminha o estudante para atendimento humano do ASA.
- Nenhuma resposta do agente executa automaticamente uma ação com impacto acadêmico, financeiro ou disciplinar; o agente apenas orienta e recomenda.
- Toda resposta deve indicar a fonte do documento e a data de atualização utilizada, quando disponível.

2.4 Critérios de aceitação (MVP da Entrega 1)

- Dado um conjunto de perguntas de teste, o endpoint `/api/perguntar` retorna resposta, evidências e grau de confiança para cada uma.
- Para perguntas fora da base de conhecimento, o agente se abstém corretamente em vez de responder com base incerta.
- A base de conhecimento contém os 20 documentos fictícios cadastrados (`seed_v2.sql`), catalogados com categoria, fonte e data de atualização, cobrindo os principais temas de atendimento do ASA.
- O mock de dados acadêmicos (TOTVS) responde a uma consulta de estudante fictício com dados de situação acadêmica e financeira coerentes.
- O notebook de Álgebra Linear demonstra, com dados reais do projeto, ao menos duas operações (similaridade de cosseno e uma operação adicional) e explica o resultado.

2.5 Riscos éticos identificados

Risco	Descrição	Mitigação
Alucinação de procedimento	O agente pode gerar uma resposta plausível, mas incorreta, sobre prazos ou regras.	Resposta restrita ao conteúdo recuperado (RAG) e mecanismo de abstenção por limiar de confiança.
Exposição indevida de dados financeiros/acadêmicos	Consultas ao TOTVS/mock podem expor mais dado do que o necessário para responder à dúvida.	Minimização de dados: só os campos necessários para a pergunta específica são consultados.
Viés no reconhecimento da pergunta	O modelo de embeddings (<code>bge-small-en-v1.5</code>) é otimizado para inglês; a qualidade semântica em português, idioma de todo o conteúdo do ASA, fica abaixo do ideal em relação a um modelo nativo em português.	Dicionário de sinônimos em português complementando a busca; fallback por TF-IDF; testes com perguntas de diferentes formulações; avaliação de modelos multilíngues como evolução futura.
Uso do histórico para fins não previstos	Registros de conversa podem ser usados fora da finalidade de suporte ao estudante.	Finalidade explícita, acesso controlado por perfil e retenção mínima necessária.

2.6 Plano de proteção de dados

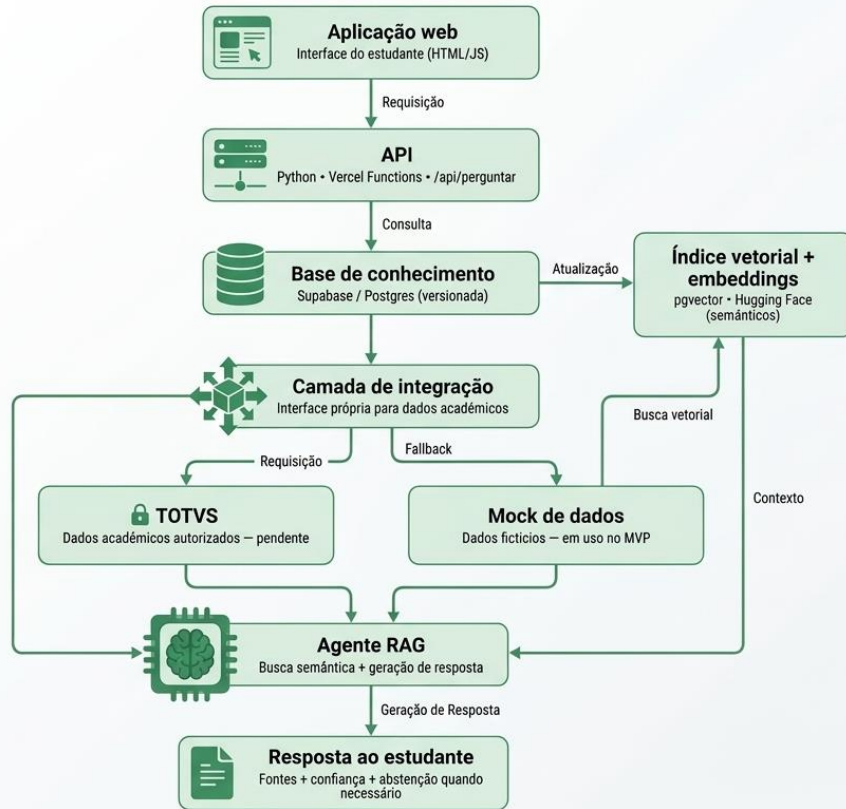
- Nenhuma credencial, token ou chave de serviço é versionada no repositório; uso de variáveis de ambiente (`.env`, Vercel Environment Variables).
- Dados de estudantes usados no desenvolvimento e na demonstração são fictícios.
- A chave de serviço do Supabase (`service_role`) é usada apenas no backend; o frontend nunca recebe essa chave.
- Row Level Security (RLS) habilitado em todas as tabelas com dado pessoal ou acadêmico sensível: o estudante só acessa os próprios dados, e perfis com maior privilégio (atendente, administrador, equipe técnica) têm acesso ampliado conforme política definida no banco (ver seção 3.3).
- O endereço do estudante é armazenado sempre de forma aproximada (ex.: região da cidade), nunca exato, e valores monetários exatos da situação financeira não são expostos ao estudante sem autorização.
- O acesso real ao TOTVS depende de autorização formal da FECAP/ASA, ainda pendente; enquanto isso, todo o desenvolvimento segue por um mock com dados fictícios que implementa o mesmo contrato.
- Quando dados reais forem incorporados, será aplicada minimização (apenas campos necessários) e controle de acesso por perfil.

3. Arquitetura preliminar, modelo de dados e contratos

3.1 Visão geral da arquitetura

A solução segue uma arquitetura web + API + dados + agente + nuvem, substituindo o aplicativo mobile do enunciado original por uma aplicação web, decisão validada pelo grupo para priorizar a integração com o TOTVS e agilizar o desenvolvimento no semestre.

Fluxo de Sistema: Assistente de IA Acadêmico



Toda a infraestrutura roda em nuvem: a API é publicada como funções serverless na Vercel e o banco de dados (relacional + vetorial) roda no Supabase (Postgres com extensão pgvector).

3.2 Componentes

Camada	Componente	Tecnologia
Aplicação	Interface web do estudante (chat multi-conversa, evidências, encaminhamento humano)	HTML/JS/CSS, sem framework, consumindo /api/perguntar
API	Endpoint de pergunta e resposta	Python, função serverless na Vercel (/api/perguntar)
Vetorização	Geração de embeddings da pergunta (em tempo real, no endpoint) e dos chunks da base de conhecimento (em lote, via script gerar_embeddings.py)	Hugging Face Inference API (BAAI/bge-small-en-v1.5, 384 dimensões), com fallback local por hashing + TF-IDF (tabela de IDF compartilhada via idf_table.json)
Dados	Documentos oficiais, chunks, estudantes, situação acadêmica/financeira, conversas	Supabase (Postgres + pgvector)
Busca semântica	Recuperação dos trechos mais relevantes	Função SQL buscar_chunks_similares (RPC no Supabase)

Camada	Componente	Tecnologia
Integração acadêmica	Situação do estudante para personalizar a resposta	Mock com dados fictícios, mesmo contrato previsto para o TOTVS
Nuvem	Hospedagem da API e do frontend	Vercel

Nota sobre a escolha do modelo de embeddings: o modelo originalmente planejado (sentence-transformers/all-MiniLM-L6-v2) foi descontinuado pela Hugging Face para extração de vetor na API gratuita, passando a ser servido apenas para comparação direta de frases. O grupo migrou para BAAI/bge-small-en-v1.5, que mantém a mesma dimensão (384) e está confirmado funcionando; por ser otimizado para inglês, a qualidade semântica em português fica abaixo do ideal (ver risco na seção 2.5), mas ainda supera o método de hashing + TF-IDF usado como fallback.

3.3 Modelo de dados (principais entidades)

O schema está implementado no Supabase (Postgres) com tipos enumerados para os campos controlados (papel do usuário, canal, status de conversa, decisão de validação, prioridade, status de matrícula e categoria de score), e está organizado nos seguintes grupos:

Perfis e dados do estudante (RF01/RF02)

Entidade	Campos principais	Finalidade
perfis	papel (estudante/atendente/administrador/equipe_tecnica), nome_completo, email, ativo	Estende auth.users do Supabase; controla o perfil de acesso conforme RF01.
estudantes	ra, curso, semestre_atual, status (regular/trancado/inadimplente/formado/evadido), endereco_aproximado	Dados do estudante; endereço é sempre aproximado (ex.: “Zona Sul – SP”), nunca exato, conforme regra do edital.
situacao_financeira	adimplente, possui_acordo, quantidade_pendencias, tipo_financiamento	Contexto financeiro; valores monetários exatos nunca são expostos ao estudante sem autorização.
situacao_academica	media_geral, percentual_frequencia, quantidade_reprovacoes, tendencia_desempenho	Contexto acadêmico usado para personalizar a orientação.
matriculas	periodo_letivo, status, data_solicitacao, data_confirmacao	Histórico de matrícula do estudante por período letivo.

Base de conhecimento e busca semântica (RF16)

Entidade	Campos principais	Finalidade
documentos_oficiais	titulo, categoria, fonte, versao, ativo, data_atualizacao, criado_por	Catálogo dos documentos institucionais, versionado e com possibilidade de desativação sem exclusão.
documentos_chunks	documento_id, ordem, conteudo, embedding (vetor de 384 dimensões)	Trechos vetorizados dos documentos, indexados com pgvector (ivfflat, similaridade de cosseno) para a busca semântica.

Conversas, respostas e evidências (RF06/RF10)

Entidade	Campos principais	Finalidade
conversas	estudante_id, canal, status (aberta/encerrada/encaminhada_humano)	Sessão de interação do estudante com o agente.
mensagens	conversa_id, remetente (estudante/agente/atendente), conteudo	Histórico de mensagens trocadas dentro de cada conversa.
respostas_agente	resposta_gerada, confianca_score, versao_modelo, abstencao	Resposta gerada pelo agente, com grau de confiança e sinalização de abstenção.
resposta_evidencias	resposta_id, chunk_id, similaridade, posicao_ranking	Evidências (trechos-fonte) que sustentam cada resposta, em ordem de relevância.

Validação humana e encaminhamento (RF07/RF08/RF09)

Entidade	Campos principais	Finalidade
validacoes_humanas	atendente_id, decisao (confirmado/ajustado/rejeitado/pendente), justificativa	Registro da revisão humana sobre a resposta do agente, com justificativa obrigatória.
encaminhamentos	motivo, setor_destino, prioridade, atendente_responsavel, status, prazo	Fila de casos encaminhados ao ASA quando o agente se abstém ou identifica necessidade de atendimento humano.

Auditoria e melhoria contínua (RF10/RF17)

Entidade	Campos principais	Finalidade
auditoria_logs	usuario_id, acao, entidade, entidade_id, detalhes (jsonb)	Log de ações relevantes do sistema (resposta gerada, validação humana, login), para rastreabilidade.
feedback_respostas	estudante_id, avaliacao (util/nao_util), comentario	Avaliação do estudante sobre a utilidade da resposta recebida, usada para melhoria do agente.

Extensão opcional

Entidade	Campos principais	Finalidade
alertas_evasao	score_financeiro, score_academico, score_relacionamento, score_geral, categoria (verde/amarelo/vermelho), fatores (jsonb)	Estrutura preparada para, opcionalmente, demonstrar integração com lógica do Agente de Monitoramento como funcionalidade extra do edital.

Segurança de acesso aos dados

Todas as tabelas com dado pessoal ou acadêmico sensível (perfis, estudantes, situacao_financeira, situacao_academica, conversas, mensagens, respostas_agente, validacoes_humanas, auditoria_logs) têm Row Level Security (RLS) habilitado no Postgres. As políticas garantem que um estudante só enxerga os próprios dados (perfil, situação financeira/acadêmica, conversas e mensagens), enquanto atendentes, administradores e equipe técnica têm acesso ampliado conforme o perfil. Isso implementa, no nível do banco de dados, o princípio do menor privilégio exigido pelo RF01 e pelos requisitos de privacidade do edital.

Pipeline de ingestão da base de conhecimento

A população da base de conhecimento segue um pipeline em três etapas, já executado uma vez sobre os 20 documentos fictícios desta entrega, e a ser repetido quando os documentos reais do ASA forem incorporados (Entrega 2):

1. Cadastro do documento oficial na tabela `documentos_oficiais` (título, categoria, fonte, data de atualização).
2. Divisão do documento em trechos (chunks) e inserção em `documentos_chunks`, ainda sem embedding.
3. Execução do script `gerar_embeddings.py`, que busca todos os chunks sem processar, calcula a tabela de IDF do corpus inteiro (salva em `idf_table.json`, que precisa ser versionado no repositório) e gera o embedding semântico de cada chunk via Hugging Face, atualizando a coluna `embedding` no Supabase.

Esse script precisa ser executado novamente sempre que a base de conhecimento for atualizada (documento novo, editado ou desativado), para manter os embeddings e a tabela de IDF sincronizados com o conteúdo atual.

3.4 Contrato principal: POST /api/perguntar

Requisição:

```
{ "pergunta": "como eu tranco minha matrícula?" }
```

Resposta:

```
{ "resposta": "...", "confianca": 0.83, "abstencao": false, "evidencias":
[ { "titulo": "...", "fonte": "...", "similaridade": 0.83, "atualizado_em":
"...", "trecho": "..." } ] }
```

Quando a similaridade da melhor evidência fica abaixo do limiar de confiança definido, o campo `abstencao` vem como `true` e a resposta orienta o estudante a procurar atendimento humano do ASA, mantendo as evidências disponíveis para transparência.

3.5 Contrato da integração com o TOTVS (mock)

Como o acesso ao ambiente real do TOTVS ainda depende de autorização da FECAP/ASA, a camada de integração foi desenhada de forma isolada atrás de uma interface própria: o agente consome um contrato interno (situação acadêmica e financeira por RA), e não o TOTVS diretamente. No MVP, esse contrato é atendido por um mock com dados fictícios que devolve o mesmo formato esperado do sistema real, permitindo trocar a implementação sem alterar o agente.

4. Backlog, cronograma, responsabilidades, métricas e validação

4.1 Backlog realizado na Entrega 1

Item	Descrição	Responsável
Definir problema e valor esperado	Levantamento do problema do ASA, dos usuários e do valor esperado da solução, orientando o escopo do MVP.	VIVIAN KAORI UMAKI
Levantar requisitos funcionais e não funcionais	Definição dos RFs e RNFs do agente selecionado, alinhados ao backlog do edital.	VIVIAN KAORI UMAKI
Desenhar arquitetura preliminar	Desenho da arquitetura web + API + dados + agente + nuvem (Vercel/Supabase).	YANKO LEE
Levantar permissões e privacidade	Mapeamento de regras de acesso, LGPD e minimização de dados aplicáveis ao agente.	BRUNO EDUARDO DA SILVA PINHO

Item	Descrição	Responsável
Configurar ambiente do repositório	Setup do repositório no GitHub/GitHub Project, estrutura de pastas e ambiente Linux inicial.	ERIC VICTOR BITTU SILVA
Definir regra de negócio	Definição das regras que orientam as respostas do agente, como abstenção por baixa confiança e citação obrigatória de fonte.	VIVIAN KAORI UMAKI
Gerar embeddings	Execução do script gerar_embeddings.py para vetorizar os chunks da base de conhecimento.	YANKO LEE
Definir métricas de avaliação	Escolha das métricas de avaliação do agente: cobertura, groundedness, taxa de abstenção correta e tempo de resposta.	VIVIAN KAORI UMAKI
Empacotar API com Docker	Containerização do backend para execução reproduzível em qualquer ambiente.	ERIC VICTOR BITTU SILVA
Banco de dados inicial	Modelagem e criação do schema inicial no Supabase (Postgres + extensão pgvector).	YANKO LEE
API inicial	Implementação inicial do endpoint POST /api/perguntar.	VIVIAN KAORI UMAKI
Primeira versão executável	Integração de ponta a ponta funcionando localmente: interface, API e base de conhecimento.	ERIC VICTOR BITTU SILVA
Relatório técnico	Redação do relatório técnico e consolidação da documentação da Entrega 1.	ERIC VICTOR BITTU SILVA
Configurar logs e métricas iniciais	Implementação de logging básico e das primeiras métricas de uso e desempenho do serviço.	BRUNO EDUARDO DA SILVA PINHO
Gestão de segredos	Definição e aplicação da política de variáveis de ambiente e credenciais (.env, .gitignore).	BRUNO EDUARDO DA SILVA PINHO
Definir usuários e personas	Mapeamento dos perfis de usuário (estudante, atendente do ASA, administrador) e suas necessidades.	BRUNO EDUARDO DA SILVA PINHO
Implementar busca vetorial	Implementação da função de busca semântica (buscar_chunks_similares) via pgvector.	YANKO LEE

Observação: os itens acima refletem o trabalho efetivamente realizado até a Entrega 1, com o responsável principal de cada frente.

4.2 Cronograma (conforme etapas do enunciado)

Etapas	Período	Atividades e resultados esperados
1. Descoberta e escolha	Semanas 1-2	Entendimento do ASA, seleção do agente, usuários, problema, jornada, escopo do MVP, riscos, backlog e critérios de aceitação.
2. Dados, conhecimento e requisitos	Semanas 3-4	Mapeamento de fontes, permissões, base de conhecimento, regras, dicionário, requisitos, privacidade e arquitetura inicial.
3. Representação matemática e baseline	Semanas 5-6	Vetores, matrizes, normalização, métricas e primeira versão do modelo/RAG.
4. Protótipo web e serviços	Semanas 7-8	Aplicativo inicial, API, banco, autenticação, primeiro fluxo do agente e testes locais.

Etapa	Período	Atividades e resultados esperados
5. Nuvem e integração	Semanas 9-10	Contêiner, ambiente Linux/nuvem, integração web-API-agente, logs e primeiras métricas.
6. Funcionalidades finais	Semanas 11-12	Explicações, fila/alertas/chat, validação humana, histórico, dashboard e ajustes do modelo.
7. Avaliação e publicação	Semanas 13-14	Testes funcionais, desempenho, segurança, acessibilidade, equidade, observabilidade, deploy e validação com o parceiro.
8. Finalização	Semanas 15-16	Relatório final, documentação de extensão, banner, ensaio do pitch, correções e apresentação final.

4.3 Métricas de sucesso

- **Taxa de resposta correta:** proporção de perguntas de teste em que o agente recupera o documento/trecho correto.
- **Cobertura:** proporção de perguntas do conjunto de teste que encontram evidência suficiente (não abstenção) na base de conhecimento.
- **Groundedness:** grau em que a resposta gerada é sustentada pelo trecho recuperado, sem extrapolação.
- **Taxa de abstenção correta:** proporção de perguntas fora de escopo em que o agente corretamente se abstém em vez de responder incerto.
- **Tempo de resposta:** tempo entre o envio da pergunta e o retorno da resposta pela API.

4.4 Plano de validação

1. Montar um conjunto de perguntas de teste cobrindo os principais temas da base de conhecimento (ex.: trancamento, financeiro, TCC, biblioteca).
2. Executar cada pergunta contra o endpoint /api/perguntar e registrar resposta, confiança e evidências retornadas.
3. Comparar a resposta obtida com a resposta esperada (definida manualmente a partir da fonte oficial), classificando como correta, parcialmente correta ou incorreta.
4. Incluir perguntas propositalmente fora do escopo da base, para verificar se o agente se abstém corretamente.
5. Registrar os resultados no relatório técnico e no Model Card, junto com limitações observadas.

5. Prova de conceito do agente e protótipo do aplicativo

5.1 O que a PoC demonstra

A prova de conceito demonstra o fluxo mínimo de ponta a ponta do agente: recebimento da pergunta, busca semântica na base de conhecimento (com fallback automático caso o serviço de embeddings externo falhe), retorno de resposta com evidências e abstenção quando a confiança é insuficiente.

5.2 Cenários de teste

Os cenários abaixo usam os 20 documentos já cadastrados em seed_v2.sql (base de conhecimento fictícia, mas com conteúdo plausível e alinhado ao vocabulário reconhecido pelo agente). A coluna “documento-fonte esperado” indica qual registro de documentos_oficiais deve aparecer nas evidências da resposta.

Cenário	Pergunta (exemplo)	Documento-fonte esperado	Resultado esperado
Trancamento de matrícula	"Como eu tranco minha matrícula?"	Trancamento de Matrícula — Regulamento Acadêmico, Art. 27	Resposta citando prazo de 15 dias de antecedência e o impacto em bolsas/financiamentos, com fonte e data de atualização.
Questão financeira	"Tem algum auxílio para quem está com dificuldade financeira?"	Auxílio Financeiro Emergencial — Manual do Aluno, Seção 6.4	Resposta explicando que o auxílio exige comprovação de vulnerabilidade e análise em até 10 dias úteis pelo ASA.
TCC	"Quando eu preciso começar meu TCC?"	Trabalho de Conclusão de Curso (TCC) — Manual do Aluno, Seção 7.3	Resposta indicando início no penúltimo semestre e prazo de definição do orientador.
Biblioteca	"Quantos livros eu posso pegar emprestado?"	Biblioteca - Empréstimo de Livros — Manual do Aluno, Seção 8.1	Resposta citando o limite de 3 exemplares por 14 dias, com renovação automática se não houver fila.
Fora de escopo (abstenção)	"Qual o cardápio do restaurante hoje?"	Nenhum (fora da base de conhecimento)	Agente se abstém por falta de evidência suficiente e encaminha para atendimento humano do ASA.

5.3 Estado atual do protótipo

- Schema completo modelado e aplicado no Supabase: perfis e perfis de acesso, dados do estudante (estudantes, situacao_financeira, situacao_academica, matriculas), base de conhecimento (documentos_oficiais, documentos_chunks), conversas e mensagens, respostas do agente com evidências, validação humana, encaminhamentos, auditoria e feedback.
- Base de conhecimento fictícia populada (seed_v2.sql): 20 documentos oficiais cadastrados (rematrícula, trancamento, TCC, estágio, biblioteca, auxílio financeiro, entre outros), cada um com um chunk de conteúdo, já vetorizado.
- Script gerar_embeddings.py executado sobre os 20 chunks fictícios: tabela de IDF calculada e coluna embedding preenchida via Hugging Face para todos os chunks da base.
- Usuários fictícios criados no Supabase Auth e mock de estudante populado (perfis, estudantes, situacao_academica, situacao_financeira), permitindo consultas de situação acadêmica/financeira por RA.
- Row Level Security (RLS) habilitado nas tabelas sensíveis, com políticas de acesso por perfil já definidas.
- Função de busca vetorial (buscar_chunks_similares) implementada via RPC no Postgres, usando pgvector.
- Endpoint POST /api/perguntar implementado em Python, publicado como função serverless na Vercel, com geração de embeddings semânticos via Hugging Face e fallback automático por hashing + TF-IDF.
- Mecanismo de abstenção por limiar de confiança já implementado no código da API.

- Interface web (index.html) funcional, chamando o endpoint /api/perguntar de verdade: suporta múltiplas conversas em paralelo (sidebar), sugestões de pergunta, indicador de “digitando”, exibição do grau de confiança, painel expansível de evidências (título, fonte, similaridade) e botão de encaminhamento para atendente humano.
- Bateria de perguntas de teste do plano de validação (seção 4.4) executada contra o endpoint publicado, incluindo cenários de abstenção para perguntas fora de escopo; resultados detalhados no relatório técnico e no Model Card.
- Planejado para a Entrega 2 (não pendente nesta etapa): coleta e ingestão dos documentos oficiais reais do ASA mais aderentes ao Agente para o Estudante, substituindo a base fictícia — ver decisão de escopo na seção 1.7.

5.4 Demonstração

Os cenários da seção 5.2 já foram executados contra o endpoint publicado como parte da bateria de validação (seção 4.4). Na apresentação da PoC, esses mesmos cenários serão repetidos ao vivo, mostrando a resposta, as evidências retornadas (fonte, trecho, similaridade) e o comportamento de abstenção no cenário fora de escopo.

6. Referências

- FECAP · Área do Sucesso Alvarista (ASA) — <https://www.fecap.br/asa/>
- Enunciado do Projeto Interdisciplinar · ASA · 5º CCOMP · 2º semestre de 2026 (versão FINAL)
- Repositório do projeto — <https://github.com/2026-2-NCC5/Projeto3>